

实验 3：支持向量机与 Logistic 回归分类算法实现及验证

一：实验目的

- 1、了解支持向量机和 Logistic 回归算法的概念；
- 2、掌握支持向量机和 Logistic 回归算法的程序流程；
- 3、理解支持向量机和 Logistic 回归算法的一般应用。

二：实验环境

- 1、系统：windows 7、windows 10 或 windows 11
- 2、软件：PyCharm

三：实验内容

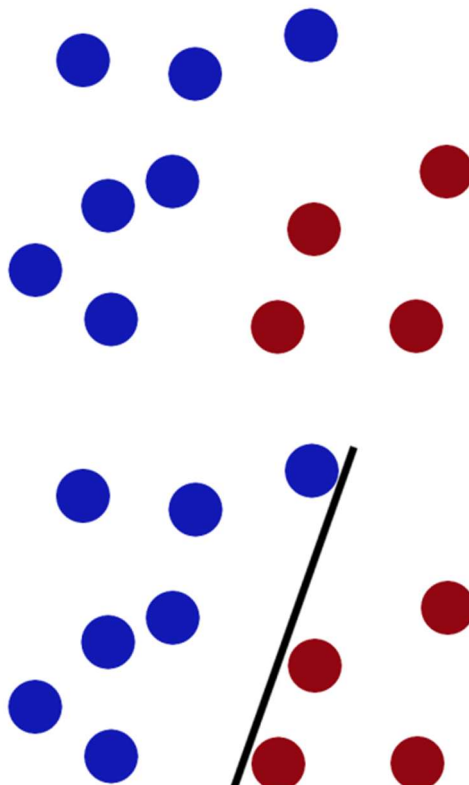
（一）支持向量机分类实验

1、什么是 SVM?

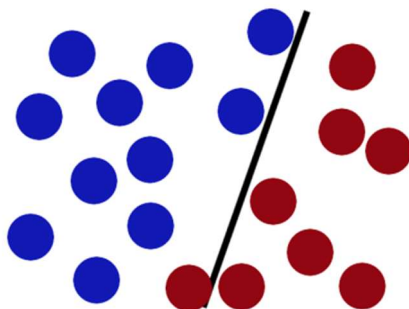
SVM 的英文全称是 Support Vector Machines，我们叫它支持向量机。支持向量机是我们用于分类的一种算法。

看一个实例，桌子上有规律放了两色颜色的球，要求用一条直线将两色球按颜色分开，并且当放入更多球之后，这条分类线仍然适用。

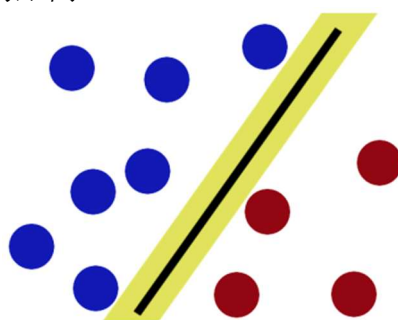
可以分为



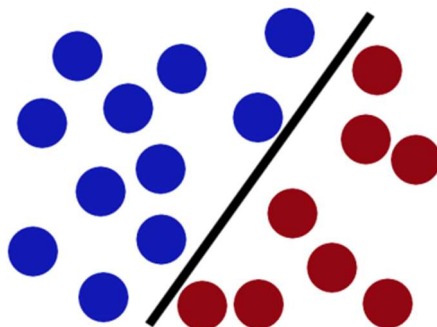
然后又在桌上放了更多的球，似乎有一个球超过了直线。显然，需要对直线做出调整。



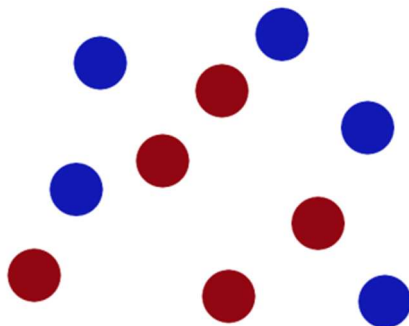
SVM 就是试图找到直线的最佳位置，好让在线的两边有尽可能大的间隙。这个间隙就是球到直线的距离。



现在，即使放了更多的球，这条线仍然是一个好的分界线。

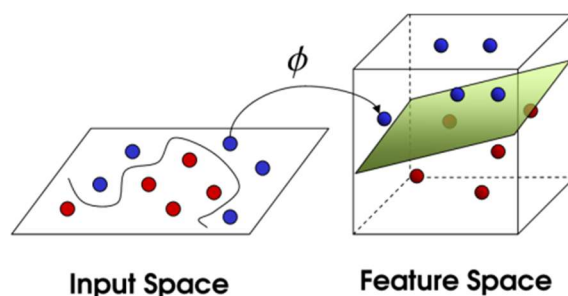


找到最佳直线之后，加大分类难度。

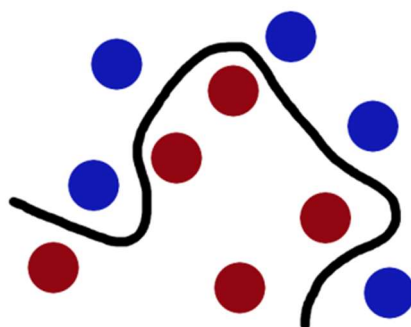


现在，没有合适的直线可以很好的分开两种球，现在怎么办呢？

所以咱们可以把这些球想象到三维空间里，就可以使用一张纸将两类球分开，就像下图所示。



现在，从空中角度看这些球，这些球看起来像是被一条曲线分开了。

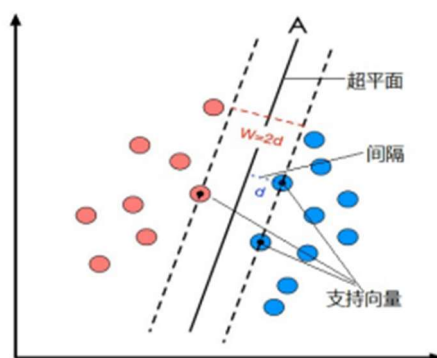


在 SVM 里，把这些球叫做 **data**（数据），把直线或平面叫做 **classifier**（分类器），找到最大间隔的 **trick**（技巧）叫做 **optimization**（优化），那张纸叫做 **hyperplane**（超平面）。

概述一下：

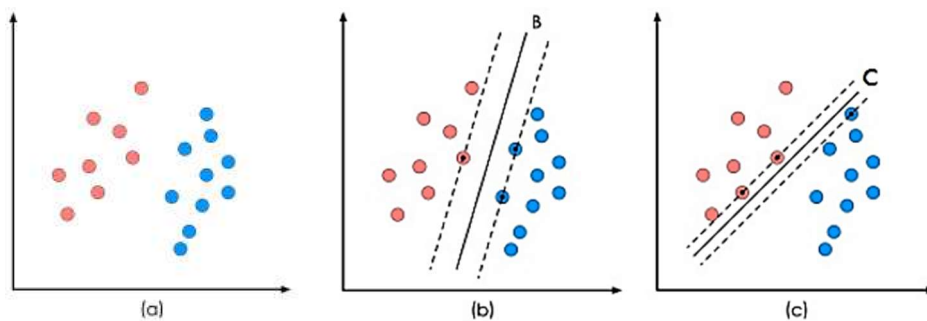
当一个分类问题，数据是**线性可分**的，也就是用一条直线就可以将两种小球分开的时候，我们只要将直线的位置放在让小球距离直线的距离最大化的位置即可，寻找这个最大间隔的过程，就叫做**最优化**。

但是，现实往往是很残酷的，一般的数据是线性不可分的，也就是找不到一条直线将两种小球很好的分类。这个时候，我们就需要用一张纸代替直线将小球进行分类。想要让数据从二维到三维，我们需要的东西就是**核函数(kernel)**，用于切分小球的纸，这张纸就是**超平面**。如果数据集是 N 维的，那么超平面就是 $N-1$ 维的。

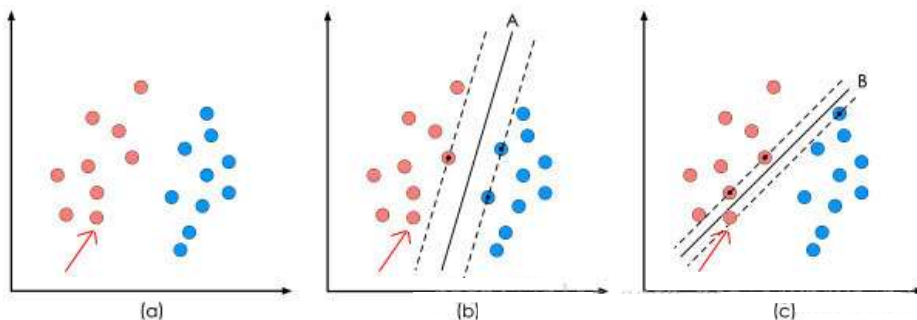


2、线性 SVM

先看下线性可分的二分类问题。



上图中的(a)是已有的数据，红色和蓝色分别代表两个不同的类别。数据显然是线性可分的，但是将两类数据点分开的直线显然不止一条。上图的(b)和(c)分别给出了 B、C 两种不同的分类方案，其中黑色实线为分界线，术语称为“决策面”。每个决策面对应了一个线性分类器。虽然从分类结果上看，分类器 A 和分类器 B 的效果是相同的。但是他们的性能是有差距的，看下图：



在"决策面"不变的情况下，又添加了一个红点。可以看到，分类器 B 依然能很好的分类结果，而分类器 C 则出现了分类错误。显然分类器 B 的"决策面"放置的位置优于分类器 C 的"决策面"放置的位置，SVM 算法也是这么认为的，它的依据就是分类器 B 的分类间隔比分类器 C 的分类间隔大。这里涉及到第一个 SVM 独有的概念"分类间隔"。

在保证决策面方向不变且不会出现错分样本的情况下移动决策面，会在原来的决策面两侧找到两个极限位置(越过该位置就会产生错分现象)，如虚线所示。

虚线的位置由决策面的方向和距离原决策面最近的几个样本的位置决定。而这两条平行虚线正中间的分界线就是在保持当前决策面方向不变的前提下的最优决策面。两条虚线之间的垂直距离就是这个最优决策面对应的分类间隔。

显然每一个可能把数据集正确分开的方向都有一个最优决策面(有些方向无论如何移动决策面的位置也不可能将两类样本完全分开)，而不同方向的最优决策面的分类间隔通常是不同的，那个具有“最大间隔”的决策面就是 SVM 要寻找的最优解。而这个真正的最优解对应的两侧虚线所穿过的样本点，就是 SVM 中的支持样本点，称为"支持向量"。支撑起超平面的样本点即为支持向量。

(1) 数学建模

求解这个"决策面"的过程，就是最优化。一个最优化问题通常有两个基本的因素：

- 1) 目标函数，也就是希望什么东西的什么指标达到最好；

2) 优化对象, 你期望通过改变哪些因素来使你的目标函数达到最优。

在线性 SVM 算法中, 目标函数显然就是那个"分类间隔", 而优化对象则是决策面。所以要对 SVM 问题进行数学建模, 首先要对上述两个对象("分类间隔"和"决策面")进行数学描述。按照一般的思维习惯, 我们先描述决策面。

数学建模的时候, 先在二维空间建模, 然后再推广到多维。

1) "决策面"方程

我们都知道二维空间下一条直线的方式如下所示:

$$y = ax + b$$

现在我们做个小小的改变, 让原来的 x 轴变成 x_1 , y 轴变成 x_2 。

$$x_2 = ax_1 + b$$

移项得:

$$ax_1 + (-1)x_2 + b = 0$$

将公式向量化得:

$$[a, -1] \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + b = 0$$

进一步向量化, 用 ω 列向量和 x 列向量和标量 b 进一步向量化:

$$\omega^T x + b = 0$$

其中, 向量 ω 和 x 分别为:

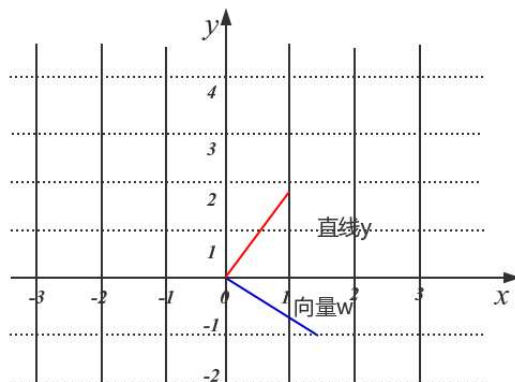
$$\omega = [\omega_1, \omega_2]^T, \quad x = [x_1, x_2]^T$$

这里 $\omega_1=a$, $\omega_2=-1$ 。我们都知道, 最初的那个直线方程 a 和 b 的几何意义, a 表示直线的斜率, b 表示截距, a 决定了直线与 x 轴正方向的夹角, b 决定了直线与 y 轴交点位置。那么向量化后的直线的 ω 和 b 的几何意义是什么呢?

现在假设:

$$\omega = [\omega_1, \omega_2]^T, \quad x = [x_1, x_2]^T$$

在坐标轴上画出直线和向量 ω :



蓝色的线代表向量 ω ，红色的线代表直线 y 。我们可以看到向量 ω 和直线的关系为垂直关系。这说明了向量 ω 也控制这直线的方向，只不过是和这个直线的方向是垂直的。标量 b 的作用也没有变，依然决定了直线的截距。此时，我们称 ω 为直线的法向量。

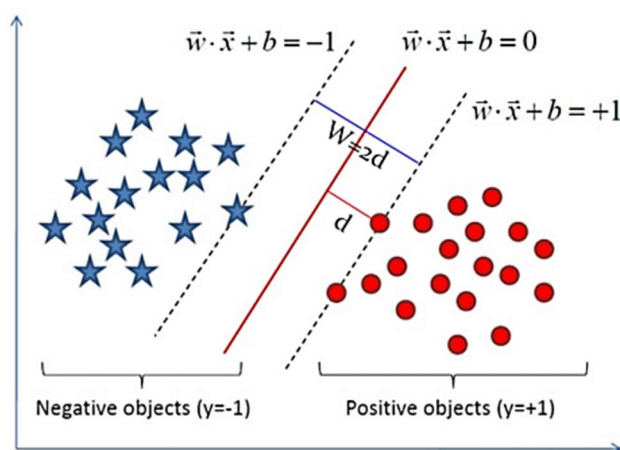
二维空间的直线方程已经推导完成，将其推广到 n 为空间，就变成了超平面方程。(一个超平面，在二维空间的例子就是一个直线)但是它的公式没变，依然是：

$$\omega^T x + b = 0$$

我们已经顺利推导出了"决策面"方程，它就是我们的超平面方程，之后，我们统称其为超平面方程。

2) "分类间隔"方程

现在，我们依然对于一个二维平面的简单例子进行推导。



我们已经知道间隔的大小实际上就是支持向量对应的样本点到决策面的距离的二倍。那么图中的距离 d 我们怎么求？我们高中都学过，点到直线的距离公式如下：

$$d = \left| \frac{Ax_0 + By_0 + C}{\sqrt{A^2 + B^2}} \right|$$

公式中的直线方程为 $Ax_0 + By_0 + C = 0$ ，点 P 的坐标为 (x_0, y_0) 。

现在，将直线方程扩展到多维，求得我们现在的超平面方程，对公式进行如下变形：

$$d = \frac{|\omega^T x + b|}{\|\omega\|}$$

这个 d 就是"分类间隔"。其中 $\|\omega\|$ 表示 ω 的二范数，求所有元素的平方和，然后再开方。比如对于二维平面：

$$\omega = [\omega_1, \omega_2]^T$$

那么，

$$\|\omega\| = \sqrt{\omega_1^2 + \omega_2^2}$$

我们目的是为了找出一个分类效果好的超平面作为分类器。分类器的好坏的评定依据是分类间隔 $\Omega=2d$ 的大小，即分类间隔 Ω 越大，我们认为这个超平面的分类效果越好。此时，求解超平面的问题就变成了求解分类间隔 Ω 最大化的为题。 Ω 的最大化也就是 d 最大化。

3) 约束条件

看起来，我们已经顺利获得了目标函数的数学形式。但是为了求解 ω 的最大值。我们不得不面对如下问题：

- 我们如何判断超平面是否将样本点正确分类？
- 我们知道想求距离 d 的最大值，我们首先需要找到支持向量上的点，怎么在众多的点中选出支持向量上的点呢？

上述我们需要面对的问题就是**约束条件**，也就是说我们优化的变量 d 的取值范围受到了限制和约束。

这个二维平面上有两种点，我们分别对它们进行标记：

- 红颜色的圆点标记为 1，我们人为规定其为正样本；
- 蓝颜色的五角星标记为-1，我们人为规定其为负样本。

对每个样本点 x_i 加上一个类别标签 y_i ：

$$y_i = \begin{cases} +1 & \text{红} \\ -1 & \text{蓝} \end{cases}$$

我们就可以说：“对于存在分类间隔的两类样本点，我们一定可以找到一些超平面面，使其对于所有的样本点均满足下面的条件：

$$\begin{cases} \omega^T x_i + b \geq 1, \forall y_i = +1 \\ \omega^T x_i + b \leq -1, \forall y_i = -1 \end{cases}$$

上述方程即给出了 SVM 最优化问题的约束条件。这时候，可能有人会问了，为什么标记为 1 和-1 呢？因为这样标记方便我们将上述方程变成如下形式：

$$y_i(\omega^T x_i + b) \geq 1, \forall x_i$$

正是因为标签为 1 和-1，才方便我们将约束条件变成一个约束方程，从而方便我们的计算。

4) 线性 SVM 优化问题基本描述

现在整合一下思路，我们已经得到我们的目标函数：

$$d = \frac{|\omega^T x_i + b|}{\|\omega\|} = \frac{1}{\|\omega\|}, \forall \text{ 支持向量 } x_i$$

我们的优化目标是 d 最大化。我们已经说过，我们是用支持向量上的样本点求解 d 的最大化的问题。

因为，我们只关心支持向量上的点。随后我们求解 d 的最大化问题变成了 $\|\omega\|$ 的最小化问题。进而 $\|\omega\|$ 的最小化问题等效于

$$\min_{\omega, b} \frac{1}{2} \|\omega\|^2$$

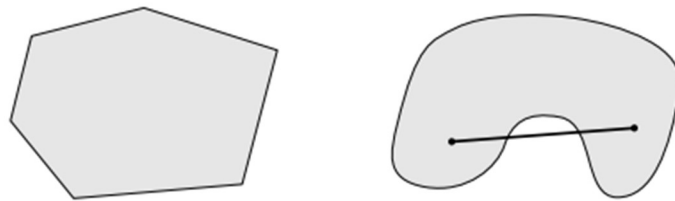
为什么要做这样的等效呢？这是为了在进行最优化的过程中对目标函数求导时比较方便，但这绝对不影响最优化问题最后的求解。我们将最终的目标函数和约束条件放在一起进行描述：

$$\begin{aligned} \min_{\omega, b} \quad & \frac{1}{2} \|\omega\|^2 \\ \text{s. t.} \quad & y_i (\omega^T x_i + b) \geq 1, \quad i = 1, 2, \dots, n \end{aligned}$$

这里 n 是样本点的总个数，缩写 s.t. 表示 "Subject to"，是 "服从某某条件" 的意思。上述公式描述的是一个典型的不等式约束条件下的二次型函数优化问题，同时也是支持向量机的基本数学模型。

5) 求解准备

我们已经得到支持向量机的基本数学模型，接下来的问题就是如何根据数学模型，求得我们想要的最优解。在学习求解方法之前，我们得知道一点，想用我下面讲述的求解方法有一个前提，就是我们的目标函数必须是凸函数。理解凸函数，我们还要先明确另一个概念，凸集。在凸几何中，凸集 (convex set) 是在凸组合下闭合的放射空间的子集。看一幅图可能更容易理解：



左右量图都是一个集合。如果集合中任意 2 个元素连线上的点也在集合中，那么这个集合就是凸集。显然，上图中的左图是一个凸集，上图中的右图是一个非凸集。

凸函数的定义也是如此，其几何意义表示为函数任意两点连线上的值大于对应自变量处的函数值。

通常我们需要求解的最优化问题有如下几类：

- 无约束优化问题，可以写为：

$$\min f(x)$$

■ 有等式约束的优化问题，可以写为：

$$\begin{aligned} \min f(x) \\ \text{s.t. } h_i(x) = 0, \quad i = 1, 2, \dots, n \end{aligned}$$

■ 有不等式约束的优化问题，可以写为：

$$\begin{aligned} \min f(x) \\ \text{s.t. } g_i(x) \leq 0, \quad i = 1, 2, \dots, n \\ h_j(x) = 0, \quad j = 1, 2, \dots, m \end{aligned}$$

对于第(a)类的优化问题，尝尝使用的方法就是**费马大定理**(Fermat)，即使用求取函数 $f(x)$ 的导数，然后令其为零，可以求得候选最优值，再在这些候选值中验证；如果是凸函数，可以保证是最优解。这也就是我们高中经常使用的求函数的极值的方法。

对于第(b)类的优化问题，常常使用的方法就是**拉格朗日乘子法** (Lagrange Multiplier)，即把等式约束 $h_i(x)$ 用一个系数与 $f(x)$ 写为一个式子，称为拉格朗日函数，而系数称为拉格朗日乘子。通过拉格朗日函数对各个变量求导，令其为零，可以求得候选值集合，然后验证求得最优值。

对于第(c)类的优化问题，常常使用的方法就是 **KKT 条件**。同样地，我们把所有的等式、不等式约束与 $f(x)$ 写为一个式子，也叫拉格朗日函数，系数也称拉格朗日乘子，通过一些条件，可以求出最优值的必要条件，这个条件称为 **KKT 条件**。

必要条件和充要条件如果不理解，可以看下面这句话：

- A 的**必要条件**就是 A 可以推出的结论
- A 的**充分条件**就是可以推出 A 的前提

了解到这些，现在让我们再看一下我们的最优化问题：

$$\begin{aligned} \min_{\omega, b} \frac{1}{2} \|\omega\|^2 \\ \text{s.t. } y_i(\omega^T x_i + b) \geq 1, \quad i = 1, 2, \dots, n \end{aligned}$$

现在，我们的这个对优化问题属于哪一类？很显然，它属于第(c)类问题。因为，在学习求解最优化问题之前，我们还要学习两个东西：拉格朗日函数和 KKT 条件。

6) 拉格朗日函数

首先，我们先要从宏观的视野上了解一下拉格朗日对偶问题出现的**原因和背景**。

我们知道我们要求解的是最小化问题，所以一个直观的想法是如果我能够构

造一个函数，使得该函数在可求解区域内与原目标函数完全一致，而在可求解区域外的数值非常大，甚至是无穷大，那么这个**没有约束条件的新目标函数的优化问题**就与原来**有约束条件的原始目标函数的优化问题**是等价的问题。这就是使用拉格朗日方程的目的，它将**约束条件放到目标函数**中，从而将有约束优化问题转换为无约束优化问题。

随后，人们又发现，使用拉格朗日获得的函数，使用求导的方法求解依然困难。进而，需要对问题再进行一次转换，即使用一个数学技巧：**拉格朗日对偶**。

所以，显而易见的是，我们在拉格朗日优化我们的问题这个道路上，需要进行下面二个步骤：

- 将有约束的原始目标函数转换为无约束的新构造的拉格朗日目标函数
- 使用拉格朗日对偶性，将不易求解的优化问题转化为易求解的优化

下面，进行**第一步**：将有约束的原始目标函数转换为无约束的新构造的拉格朗日目标函数

公式变形如下：

$$L(\omega, b, \alpha) = \frac{1}{2} \|\omega\|^2 - \sum_{i=1}^n \alpha_i (y_i (\omega^T x_i + b) - 1)$$

其中 α_i 是拉格朗日乘子， α_i 大于等于 0，是我们构造新目标函数时引入的系数变量(我们自己设置)。又因为

$$\begin{aligned} \alpha_i &\geq 0 \\ y_i (\omega^T x_i + b) &\geq 1 \\ y_i (\omega^T x_i + b) - 1 &\geq 0 \\ \sum_{i=1}^n \alpha_i (y_i (\omega^T x_i + b) - 1) &\geq 0 \end{aligned}$$

所以我们的新目标函数为

$$\min_{\omega, b} \left[\max_{\alpha: \alpha_j \geq 0} L(\omega, b, \alpha) \right]$$

接下来，我们就可以求解拉格朗日对偶函数了，求解出来的值就是我们最优化问题的结果，也就是可以得到最大间隔。

进行**第二步**：拉格朗日对偶函数，将不易求解的优化问题转化为易求解的优化。

看一下我们的新目标函数，先求最大值，再求最小值。这样的话，我们首先就要面对带有需要求解的参数 ω 和 b 的方程，而 α_i 又是不等式约束，这个求解过程不好做。所以，我们需要使用拉格朗日函数对偶性，将最小和最大的位置交换一下，就变成了

$$\max_{\alpha: \alpha_j \geq 0} \left[\min_{\omega, b} L(\omega, b, \alpha) \right]$$

接下来，我们就可以求解拉格朗日对偶函数了，求解出来的值就是我们最优化问题的结果，也就是可以得到最大间隔。

要满足的条件：

- 首先必须满足这个优化问题是凸优化问题。
- 其次，需要满足 KKT 条件。

凸优化问题的定义是：求取最小值的目标函数为凸函数的一类优化问题。目标函数是凸函数我们已经知道，这个优化问题又是求最小值。所以我们的最优化问题就是凸优化问题。

接下里，就是探讨是否满足 KKT 条件了。

7) KKT 条件

我们已经使用拉格朗日函数对我们的目标函数进行了处理，生成了一个新的目标函数。通过一些条件，可以求出最优值的必要条件，这个条件就是接下来要说的 KKT 条件。一个最优化模型能够表示成下列标准形式：

$$\begin{aligned} \min f(\mathbf{x}) \\ \text{s.t. } h_j(\mathbf{x}) &= 0, j = 1, 2, \dots, p \\ g_k(\mathbf{x}) &\leq 0, k = 1, 2, \dots, q \\ \mathbf{x} &\in X \subset \mathbb{R}^n \end{aligned}$$

KKT 条件的全称是 Karush-Kuhn-Tucker 条件，KKT 条件是说最优值条件必须满足以下条件：

- 条件一：经过拉格朗日函数处理之后的新目标函数 $L(\omega, b, \alpha)$ 对 α 求导为零；
- 条件二： $h(\mathbf{x}) = 0$ ；
- 条件三： $\alpha^* g(\mathbf{x}) = 0$ ；

对于我们的优化问题：

$$\begin{aligned} \min_{\omega, b} \frac{1}{2} \|\omega\|^2 \\ \text{s.t. } y_i(\omega^T \mathbf{x}_i + b) \geq 1, i = 1, 2, \dots, n \end{aligned}$$

显然，条件二已经满足了。另外两个条件为啥也满足呢？

这里已经给出了很好的解释。现在，凸优化问题和 KKT 都满足了，问题转换成了对偶问题。而求解这个对偶学习问题，可以分为三个步骤：首先要让 $L(\omega, b, \alpha)$ 关于 ω 和 b 最小化，然后求对 α 的极大，最后利用 SMO 算法求解对偶问题中的拉格朗日乘子。

8) 对偶问题求解

第一步：

根据上述推导已知：

$$\min_{\omega, b} L(\omega, b, \alpha) = \min_{\omega, b} \left[\frac{1}{2} \|\omega\|^2 - \sum_{i=1}^n \alpha_i (y_i (\omega^T \mathbf{x}_i + b) - 1) \right]$$

首先固定 α ，要让 $L(\omega, b, \alpha)$ 关于 ω 和 b 最小化，我们分别对 ω 和 b 偏导数，令其等于 0，即：

$$\frac{\partial L}{\partial \omega} = 0 \implies \omega = \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i$$

$$\frac{\partial L}{\partial b} = 0 \implies \sum_{i=1}^n \alpha_i y_i = 0$$

将上述结果带回 $L(\omega, b, \alpha)$ 得到：

$$\begin{aligned} \mathcal{L}(\omega, b, \alpha) &= \frac{1}{2} \|\omega\|^2 - \sum_{i=1}^n \alpha_i [y_i (\omega^T \mathbf{x}_i + b) - 1] \\ &= \frac{1}{2} \omega^T \omega - \omega^T \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i - b \sum_{i=1}^n \alpha_i y_i + \sum_{i=1}^n \alpha_i \\ &= \frac{1}{2} \omega^T \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i - \omega^T \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i - b \cdot 0 + \sum_{i=1}^n \alpha_i \\ &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \left(\sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \right)^T \sum_{i=1}^n \alpha_i y_i \mathbf{x}_i \\ &= \sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \end{aligned}$$

从上面的最后一个式子，我们可以看出，此时的 $L(\omega, b, \alpha)$ 函数只含有一个变量，即 α_i 。

第二步：

现在内侧的最小值求解完成，我们求解外侧的最大值，从上面的式子得到

$$\begin{aligned} &\max_{\alpha: \alpha_j \geq 0} \left[\min_{\omega, b} L(\omega, b, \alpha) \right] \\ &\max_{\alpha} \left[\sum_{i=1}^n \alpha_i - \frac{1}{2} \sum_{i,j=1}^n \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j \right] \\ &s.t. \alpha_i \geq 0, i = 1, 2, \dots, n \\ &\sum_{i=1}^n \alpha_i y_i = 0 \end{aligned}$$

现在我们的优化问题变成了如上的形式。对于这个问题，我们有更高效的优化算法，即序列最小优化（SMO）算法。我们通过这个优化算法能得到 α ，再根

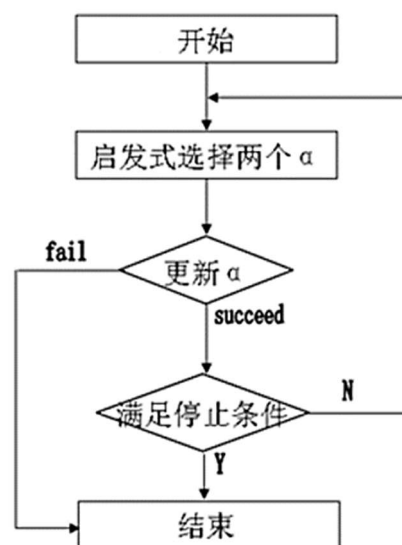
据 α ，我们就可以求解出 ω 和 b ，进而求得我们最初的目的：找到超平面，即"决策平面"。

上面我们为啥要进行推导？因为我们要将最初的原始问题，转换到可以使用 SMO 算法求解的问题，这是一种最简单直接的求解方法。

(2) SMO 算法

现在，我们已经得到了可以用 SMO 算法求解的目标函数，但是对于怎么编程实现 SMO 算法还是感觉无从下手。那么现在就聊聊如何使用 SMO 算法进行求解。

1) Platt 的 SMO 算法



1996 年，John Platt 发布了一个称为 SMO 的强大算法，用于训练 SVM。SM 表示序列最小化(Sequential Minimal Optimizaion)。Platt 的 SMO 算法是将大优化问题分解为多个小优化问题来求解的。这些小优化问题往往很容易求解，并且对它们进行顺序求解的结果与将它们作为整体来求解的结果完全一致的。在结果完全相同的同时，SMO 算法的求解时间短很多。

SMO 算法的目标是求出一系列 α 和 b ，一旦求出了这些 α ，就很容易计算出权重向量 ω 并得到分隔超平面。

SMO 算法的工作原理是：每次循环中选择两个 α 进行优化处理。一旦找到了一对合适的 α ，那么就增大其中一个同时减小另一个。这里所谓的"合适"就是指两个 α 必须符合以下两个条件，条件之一就是两个 α 必须要在间隔边界之外，而且第二个条件则是这两个 α 还没有进进行过区间化处理或者不在边界上。

2) SMO 算法的步骤：

步骤 1：计算误差：

$$E_i = f(x_i) - y_i = \left(\sum_{j=1}^n \alpha_j y_j \mathbf{x}_i^T \mathbf{x}_j + b \right) - y_i$$

步骤 2: 计算上下界 L 和 H:

$$\begin{cases} L = \max(0, \alpha_j^{old} - \alpha_i^{old}), H = \min(C, C + \alpha_j^{old} - \alpha_i^{old}) & \text{if } y_i \neq y_j \\ L = \max(0, \alpha_j^{old} + \alpha_i^{old} - C), H = \min(C, \alpha_j^{old} + \alpha_i^{old}) & \text{if } y_i = y_j \end{cases}$$

步骤 3: 计算 η :

$$\eta = \mathbf{x}_i^T \mathbf{x}_i + \mathbf{x}_j^T \mathbf{x}_j - 2\mathbf{x}_i^T \mathbf{x}_j$$

步骤 4: 更新 α_j :

$$\alpha_j^{new} = \alpha_j^{old} + \frac{y_i(E_i - E_j)}{\eta}$$

步骤 5: 根据取值范围修剪 α_j :

$$\alpha_j^{new,clipped} = \begin{cases} H, & \text{if } \alpha_j^{new} \geq H \\ \alpha_j^{new}, & \text{if } L \leq \alpha_j^{new} \leq H \\ L, & \text{if } \alpha_j^{new} \leq L \end{cases}$$

步骤 6: 更新 α_i :

$$\alpha_i^{new} = \alpha_i^{old} + y_i y_j (\alpha_j^{old} - \alpha_j^{new,clipped})$$

步骤 7: 更新 b_1 和 b_2 :

$$\begin{aligned} b_1^{new} &= b^{old} - E_i - y_i(\alpha_i^{new} - \alpha_i^{old})\mathbf{x}_i^T \mathbf{x}_i - y_j(\alpha_j^{new} - \alpha_j^{old})\mathbf{x}_j^T \mathbf{x}_i \\ b_2^{new} &= b^{old} - E_j - y_i(\alpha_i^{new} - \alpha_i^{old})\mathbf{x}_i^T \mathbf{x}_j - y_j(\alpha_j^{new} - \alpha_j^{old})\mathbf{x}_j^T \mathbf{x}_j \end{aligned}$$

步骤 8: 根据 b_1 和 b_2 更新 b :

$$b = \begin{cases} b_1 & 0 < \alpha_1^{new} < C \\ b_2 & 0 < \alpha_2^{new} < C \\ \frac{1}{2}(b_1 + b_2) & \text{otherwise} \end{cases}$$

3、编程求解线性 SVM

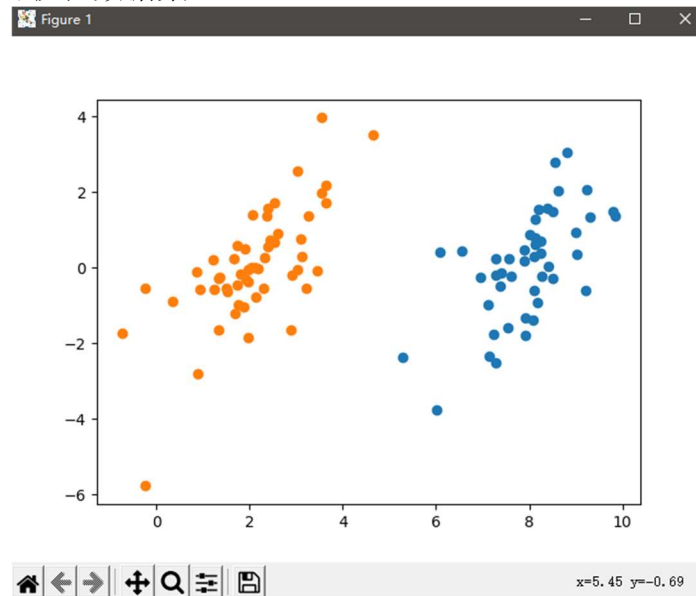
已经梳理完了 SMO 算法实现步骤，接下来按照这个思路编写代码，进行实战练习。

- ✓ 训练数据来自 testSet.txt。
- ✓ 使用简化版 SMO 算法求解线性 SVM 问题，绘制出分隔超平面（决

策面) 和相应的支持向量。

(1) 构建 SVM 分类模型

编写程序可视化数据集:



按照上述已经推导的步骤编写代码:

```
# -*- coding: UTF-8 -*-
import matplotlib.pyplot as plt
import numpy as np
import random

"""
函数说明:读取数据

Parameters:
    fileName - 文件名
Returns:
    dataMat - 数据矩阵
    labelMat - 数据标签
"""
def loadDataSet(fileName):
    dataMat = []; labelMat = []
    fr = open(fileName)
    for line in fr.readlines():
        #逐行读取, 滤除空格
        lineArr = line.strip().split("\t")
        dataMat.append([float(lineArr[0]), float(lineArr[1])])
        labelMat.append(float(lineArr[2]))
    #添加数据
    #添加标签
    return dataMat, labelMat
```



```
"""
```

函数说明:随机选择 alpha

Parameters:

i - alpha_i 的索引值

m - alpha 参数个数

Returns:

j - alpha_j 的索引值

```
"""
```

```
def selectJrand(i, m):
```

```
    j = i
```

#选择一个不等于i的j

```
    while (j == i):
```

```
        j = int(random.uniform(0, m))
```

```
    return j
```

```
"""
```

函数说明:修剪 alpha

Parameters:

aj - alpha_j 值

H - alpha 上限

L - alpha 下限

Returns:

aj - alpah 值

```
"""
```

```
def clipAlpha(aj,H,L):
```

```
    if aj > H:
```

```
        aj = H
```

```
    if L > aj:
```

```
        aj = L
```

```
    return aj
```

```
"""
```

函数说明:数据可视化

Parameters:

dataMat - 数据矩阵

labelMat - 数据标签

Returns:

无

```
"""
```

```
def showDataSet(dataMat, labelMat):
```

```
    data_plus = []
```

#正样本

```

data_minus = [] #负样本
for i in range(len(dataMat)):
    if labelMat[i] > 0:
        data_plus.append(dataMat[i])
    else:
        data_minus.append(dataMat[i])
data_plus_np = np.array(data_plus) #转换为numpy 矩阵
data_minus_np = np.array(data_minus) #转换为numpy 矩阵
plt.scatter(np.transpose(data_plus_np)[0], np.transpose(data_plus_np)[1]) #正样本散点图
plt.scatter(np.transpose(data_minus_np)[0], np.transpose(data_minus_np)[1]) #负样本散点图
plt.show()

```

"""

函数说明:简化版 SMO 算法

Parameters:

dataMatIn - 数据矩阵
 classLabels - 数据标签
 C - 松弛变量
 toler - 容错率
 maxIter - 最大迭代次数

Returns:

无

"""

```

def smoSimple(dataMatIn, classLabels, C, toler, maxIter):
    #转换为numpy 的mat 存储
    dataMatrix = np.mat(dataMatIn); labelMat = np.mat(classLabels).transpose()
    #初始化 b 参数, 统计 dataMatrix 的维度
    b = 0; m,n = np.shape(dataMatrix)
    #初始化 alpha 参数, 设为 0
    alphas = np.mat(np.zeros((m,1)))
    #初始化迭代次数
    iter_num = 0
    #最多迭代 maxIter 次
    while (iter_num < maxIter):
        alphaPairsChanged = 0
        for i in range(m):
            #步骤 1: 计算误差 Ei
            fXi = float(np.multiply(alphas,labelMat).T*(dataMatrix*dataMatrix[i,:].T)) + b
            Ei = fXi - float(labelMat[i])
            #优化 alpha, 设定一定的容错率。
            if ((labelMat[i]*Ei < -toler) and (alphas[i] < C)) or ((labelMat[i]*Ei > toler) and
            (alphas[i] > 0)):

```

```

#随机选择另一个与 alpha_i 成对优化的 alpha_j
j = selectJrand(i,m)
#步骤1: 计算误差Ej
fXj = float(np.multiply(alphas,labelMat).T*(dataMatrix*dataMatrix[j,:].T)) + b
Ej = fXj - float(labelMat[j])
#保存更新前的aplpha 值, 使用深拷贝
alphaIold = alphas[i].copy(); alphaJold = alphas[j].copy();
#步骤2: 计算上下界L 和H
if (labelMat[i] != labelMat[j]):
    L = max(0, alphas[j] - alphas[i])
    H = min(C, C + alphas[j] - alphas[i])
else:
    L = max(0, alphas[j] + alphas[i] - C)
    H = min(C, alphas[j] + alphas[i])
if L==H: print("L==H"); continue
#步骤3: 计算 eta
eta = 2.0 * dataMatrix[i,:]*dataMatrix[j,:].T - dataMatrix[i,:]*dataMatrix[i,:].T -
dataMatrix[j,:]*dataMatrix[j,:].T
if eta >= 0: print("eta>=0"); continue
#步骤4: 更新 alpha_j
alphas[j] -= labelMat[j]*(Ei - Ej)/eta
#步骤5: 修剪 alpha_j
alphas[j] = clipAlpha(alphas[j],H,L)
if (abs(alphas[j] - alphaJold) < 0.00001): print("alpha_j 变化太小"); continue
#步骤6: 更新 alpha_i
alphas[i] += labelMat[j]*labelMat[i]*(alphaJold - alphas[j])
#步骤7: 更新 b_1 和 b_2
b1 = b - Ei - labelMat[i]*(alphas[i]-alphaIold)*dataMatrix[i,:]*dataMatrix[i,:].T -
labelMat[j]*(alphas[j]-alphaJold)*dataMatrix[i,:]*dataMatrix[j,:].T
b2 = b - Ej - labelMat[i]*(alphas[i]-alphaIold)*dataMatrix[i,:]*dataMatrix[j,:].T -
labelMat[j]*(alphas[j]-alphaJold)*dataMatrix[j,:]*dataMatrix[j,:].T
#步骤8: 根据 b_1 和 b_2 更新 b
if (0 < alphas[i]) and (C > alphas[i]): b = b1
elif (0 < alphas[j]) and (C > alphas[j]): b = b2
else: b = (b1 + b2)/2.0
#统计优化次数
alphaPairsChanged += 1
#打印统计信息
print("第%d 次迭代 样本:%d, alpha 优化次数:%d" %
(iter_num,i,alphaPairsChanged))
#更新迭代次数
if (alphaPairsChanged == 0): iter_num += 1
else: iter_num = 0
print("迭代次数: %d" % iter_num)

```

```

    return b, alphas

"""
函数说明:分类结果可视化

Parameters:
    dataMat - 数据矩阵
     $\omega$  - 直线法向量
    b - 直线解决
Returns:
    无
"""

def showClassifier(dataMat,  $\omega$ , b):
    #绘制样本点
    data_plus = [] #正样本
    data_minus = [] #负样本
    for i in range(len(dataMat)):
        if labelMat[i] > 0:
            data_plus.append(dataMat[i])
        else:
            data_minus.append(dataMat[i])
    data_plus_np = np.array(data_plus) #转换为numpy 矩阵
    data_minus_np = np.array(data_minus) #转换为numpy 矩阵
    plt.scatter(np.transpose(data_plus_np)[0], np.transpose(data_plus_np)[1], s=30, alpha=0.7)
    #正样本散点图
    plt.scatter(np.transpose(data_minus_np)[0], np.transpose(data_minus_np)[1], s=30, alpha=0.7)
    #负样本散点图
    #绘制直线
    x1 = max(dataMat)[0]
    x2 = min(dataMat)[0]
    a1, a2 =  $\omega$ 
    b = float(b)
    a1 = float(a1[0])
    a2 = float(a2[0])
    y1, y2 = (-b - a1*x1)/a2, (-b - a1*x2)/a2
    plt.plot([x1, x2], [y1, y2])
    #找出支持向量点
    for i, alpha in enumerate(alphas):
        if abs(alpha) > 0:
            x, y = dataMat[i]
            plt.scatter([x], [y], s=150, c='none', alpha=0.7, linewidth=1.5, edgecolor='red')
    plt.show()

```

```
"""
```

函数说明:计算 ω

Parameters:

dataMat - 数据矩阵

labelMat - 数据标签

alphas - alphas 值

Returns:

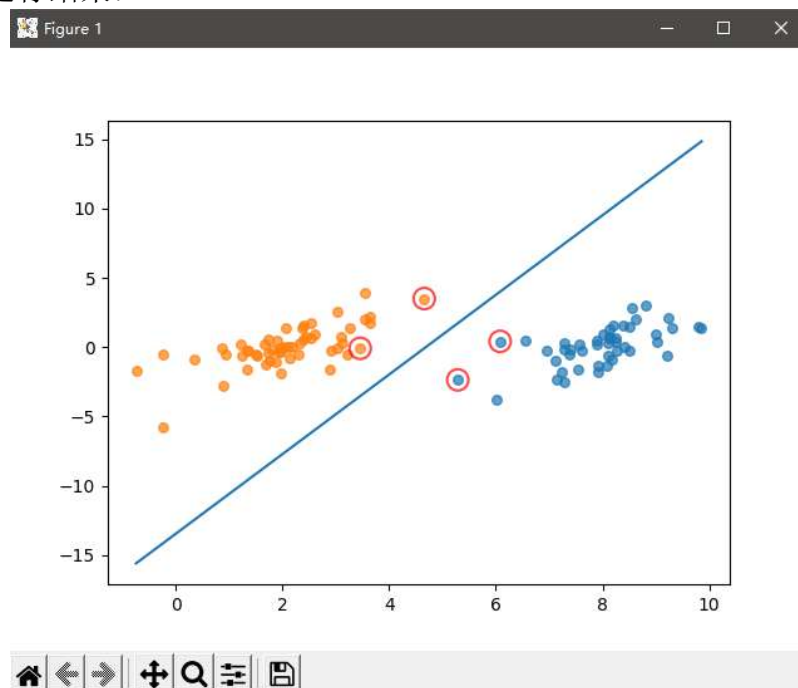
无

```
"""
```

```
def get_ω(dataMat, labelMat, alphas):  
    alphas, dataMat, labelMat = np.array(alphas), np.array(dataMat), np.array(labelMat)  
    ω = np.dot((np.tile(labelMat.reshape(1, -1).T, (1, 2)) * dataMat).T, alphas)  
    return ω.tolist()
```

```
if __name__ == '__main__':  
    dataMat, labelMat = loadDataSet('testSet.txt')  
    showDataSet(dataMat, labelMat)  
    b, alphas = smoSimple(dataMat, labelMat, 0.6, 0.001, 40)  
    ω = get_ω(dataMat, labelMat, alphas)  
    showClassifier(dataMat, ω, b)
```

程序运行结果:



其中，中间的蓝线为求出来的分类器，用红圈圈出的点为支持向量点。

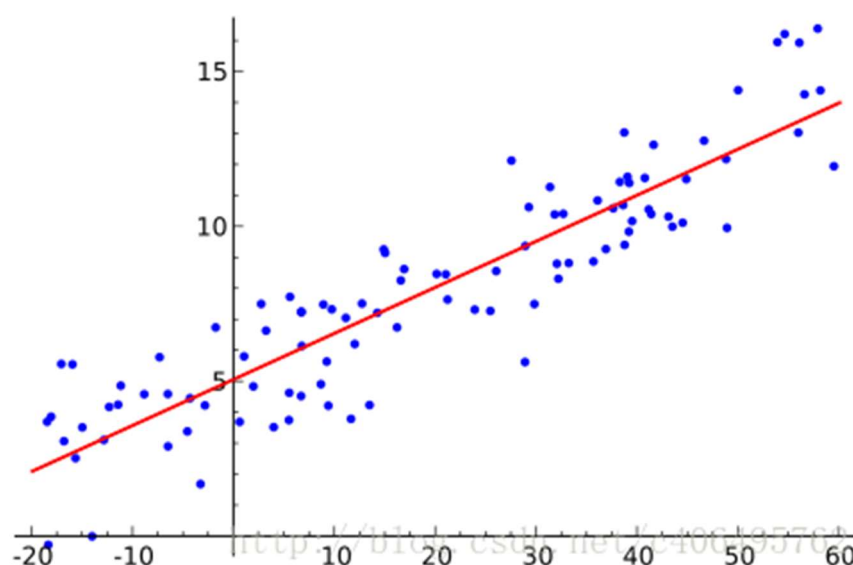
(二) Logistic 回归分类实验

1、构建 Logistic 回归分类模型

Logistic 回归是众多分类算法中的一员。通常，Logistic 回归用于二分类问题，例如预测明天是否会下雨。当然它也可以用于多分类问题。首先，让我们了解一下，什么是 Logistic 回归。

(1) Logistic 回归

假设现在有一些数据点，我们利用一条直线对这些点进行拟合(该线称为最佳拟合直线)，这个拟合过程就称作为回归，如下图所示：



Logistic 回归一种二分类算法，它利用的是 Sigmoid 函数阈值在[0,1]这个特性。Logistic 回归进行分类的主要思想是：根据现有数据对分类边界线建立回归公式，以此进行分类。其实，Logistic 本质上是一个基于条件概率的判别模型(Discriminative Model)。

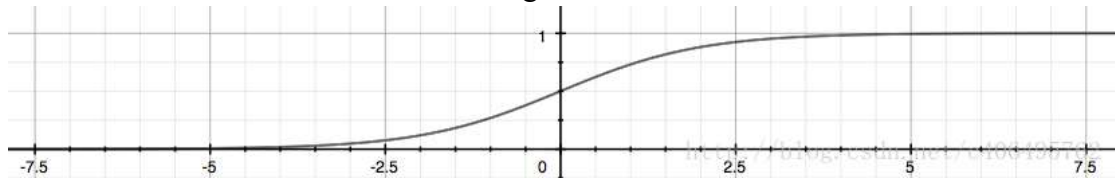
所以要想了解 Logistic 回归，我们必须先看一看 Sigmoid 函数，我们也可以称它为 Logistic 函数。它的公式如下：

$$h_{\theta}(x) = g(\theta^T x)$$
$$z = [\theta_0 \quad \theta_1 \quad \cdots \quad \theta_n] \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_n \end{bmatrix} = \theta^T x$$
$$g(z) = \frac{1}{1 + e^{-z}}$$

整合成一个公式，就变成了如下公式：

$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

下面这张图片，为我们展示了 Sigmoid 函数的样子。



z 是一个矩阵, θ 是参数列向量(要求解的), x 是样本列向量(给定的数据集)。 θ^T 表示 θ 的转置。 $g(z)$ 函数实现了任意实数到 $[0,1]$ 的映射, 这样我们的数据集 $([x_0, x_1, \dots, x_n])$, 不管是大于 1 或者小于 0, 都可以映射到 $[0,1]$ 区间进行分类。 $h_{\theta}(x)$ 给出了输出为 1 的概率。比如当 $h_{\theta}(x)=0.7$, 那么说明有 70%的概率输出为 1。输出为 0 的概率是输出为 1 的补集, 也就是 30%。

如果我们有合适的参数列向量 $\theta([\theta_0, \theta_1, \dots, \theta_n]^T)$, 以及样本列向量 $x([x_0, x_1, \dots, x_n])$, 那么我们对样本 x 分类就可以通过上述公式计算出一个概率, 如果这个概率大于 0.5, 我们就可以说样本是正样本, 否则样本是负样本。

举个例子, 对于"垃圾邮件判别问题", 对于给定的邮件(样本), 我们定义非垃圾邮件为正类, 垃圾邮件为负类。我们通过计算出的概率值即可判定邮件是否是垃圾邮件。如何得到合适的参数向量 θ ?

根据 sigmoid 函数的特性, 我们可以做出如下的假设:

$$P(y=1|x;\theta) = h_{\theta}(x)$$

$$P(y=0|x;\theta) = 1 - h_{\theta}(x)$$

上式即为在已知样本 x 和参数 θ 的情况下, 样本 x 属性正样本($y=1$)和负样本($y=0$)的条件概率。理想状态下, 根据上述公式, 求出各个点的概率均为 1, 也就是完全分类都正确。但是考虑到实际情况, 样本点的概率越接近于 1, 其分类效果越好。比如一个样本属于正样本的概率为 0.51, 那么我们就可以说明这个样本属于正样本。另一个样本属于正样本的概率为 0.99, 那么我们也可以说明这个样本属于正样本。但是显然, 第二个样本概率更高, 更具说服力。我们可以把上述两个概率公式合二为一:

$$\text{cost}(h_{\theta}(x), y) = h_{\theta}(x)^y (1 - h_{\theta}(x))^{(1-y)}$$

合并出来的 Cost, 我们称之为损失函数。当 y 等于 1 时, $(1-y)$ 项(第二项)为 0; 当 y 等于 0 时, y 项(第一项)为 0。为了简化问题, 我们对整个表达式求对数, (将指数问题对数化是处理数学问题常见的方法):

$$\text{cost}(h_{\theta}(x), y) = y \log h_{\theta}(x) + (1 - y) \log(1 - h_{\theta}(x))$$

这个损失函数，是对于一个样本而言的。给定一个样本，我们就可以通过这个损失函数求出，样本所属类别的概率，而这个概率越大越好，所以也就是求解这个损失函数的最大值。既然概率出来了，那么最大似然估计也该出场了。假定样本与样本之间相互独立，那么整个样本集生成的概率即为所有样本生成概率的乘积，再将公式对数化，便可得到如下公式：

$$J(\theta) = \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

其中， m 为样本的总数， $y(i)$ 表示第 i 个样本的类别， $x(i)$ 表示第 i 个样本，需要注意的是 θ 是多维向量， $x(i)$ 也是多维向量。

综上所述，满足 $J(\theta)$ 的最大的 θ 值即是我们要求解的模型。

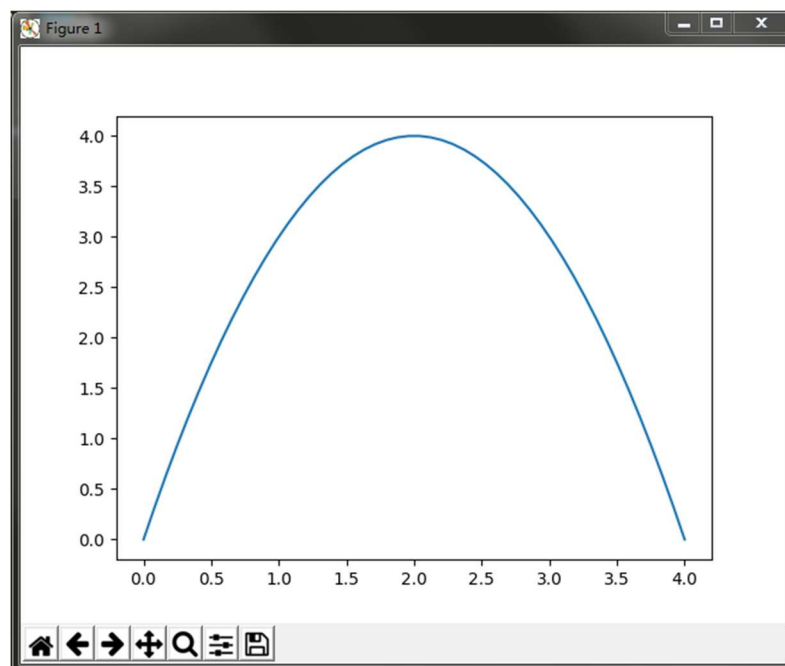
怎么求解使 $J(\theta)$ 最大的 θ 值呢？因为是求最大值，所以我们需要使用梯度上升算法。如果面对的问题是求解使 $J(\theta)$ 最小的 θ 值，那么我们就需要使用梯度下降算法。面对我们这个问题，如果使 $J(\theta) := -J(\theta)$ ，那么问题就从求极大值转换成求极小值了，使用的算法就从梯度上升算法变成了梯度下降算法，它们的思想都是相同的，学会其一，就也会了另一个。本文使用梯度上升算法进行求解。

（2）梯度上升算法

说了半天，梯度上升算法又是啥？ $J(\theta)$ 太复杂，我们先看个简单的求极大值的例子。一个看了就会想到高中生活的函数：

$$f(x) = -x^2 + 4x$$

这个函数的极值怎么求？显然这个函数开口向下，存在极大值，它的函数图像为：



求极值，先求函数的导数：

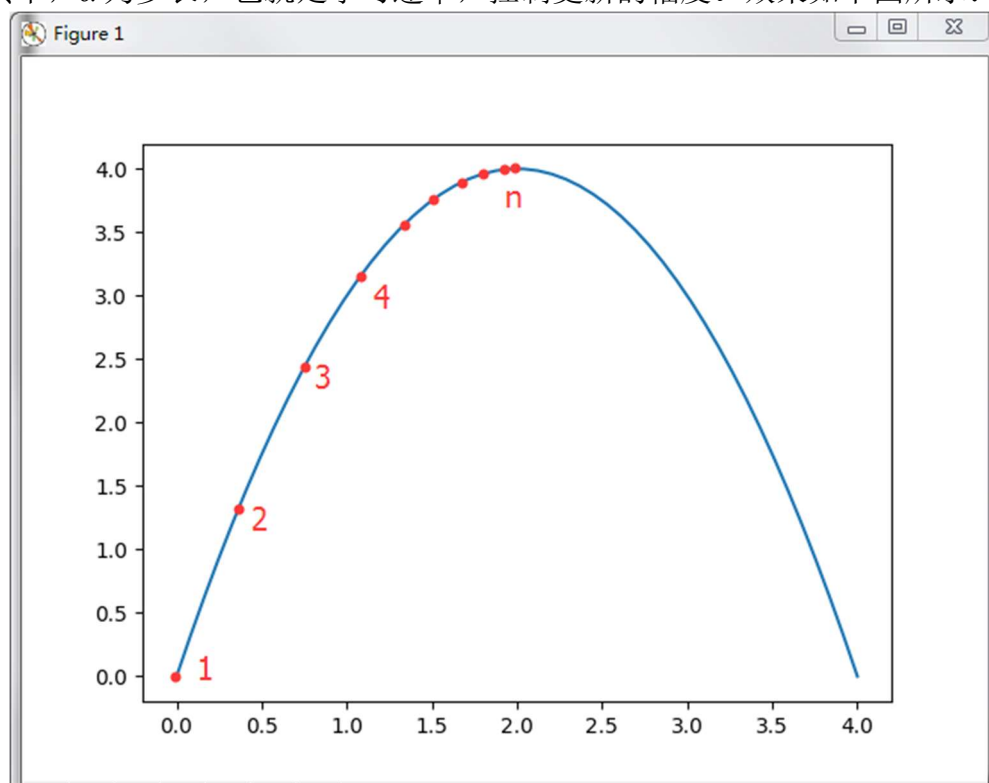
$$f'(x) = -2x + 4$$

令导数为 0，可求出 $x=2$ 即取得函数 $f(x)$ 的极大值。极大值等于 $f(2)=4$

但是真实环境中的函数不会像上面这么简单，就算求出了函数的导数，也很难精确计算出函数的极值。此时我们就可以用迭代的方法来做。就像爬坡一样，一点一点逼近极值。这种寻找最佳拟合参数的方法，就是最优化算法。爬坡这个动作作用数学公式表达即为：

$$x_{i+1} = x_i + \alpha \frac{\partial f(x_i)}{\partial x_i}$$

其中， α 为步长，也就是学习速率，控制更新的幅度。效果如下图所示：



比如从(0,0)开始，迭代路径就是 $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow \dots \rightarrow n$ ，直到求出的 x 为函数极大值的近似值，停止迭代。那么同理， $J(\theta)$ 这个函数的极值，也可以这么求解。公式可以这么写：

$$\theta_j := \theta_j + \alpha \frac{\partial J(\theta)}{\partial \theta_j}$$

由上小节可知 $J(\theta)$ 为：

$$J(\theta) = \sum_{i=1}^m [y^{(i)} \log(h_{\theta}(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_{\theta}(x^{(i)}))]$$

sigmoid 函数为：

$$h_{\theta}(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

那么，现在我只要求出 $J(\theta)$ 的偏导，就可以利用梯度上升算法，求解 $J(\theta)$ 的极大值了。

那么现在开始求解 $J(\theta)$ 对 θ 的偏导，求解如下(数学推导)：

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{\partial J(\theta)}{\partial g(\theta^T x)} \times \frac{\partial g(\theta^T x)}{\partial \theta^T x} \times \frac{\partial \theta^T x}{\partial \theta_j}$$

其中：

$$\frac{\partial J(\theta)}{\partial g(\theta^T x)} = y \times \frac{1}{g(\theta^T x)} + (y-1) \times \frac{1}{1-g(\theta^T x)}$$

再由：

$$\begin{aligned} g'(z) &= \frac{d}{dz} \frac{1}{1+e^{-z}} = \frac{1}{(1+e^{-z})^2} (e^{-z}) \\ &= \frac{1}{1+e^{-z}} \left(1 - \frac{1}{1+e^{-z}}\right) = g(z)(1-g(z)) \end{aligned}$$

可得：

$$\frac{\partial g(\theta^T x)}{\partial \theta^T x} = g(\theta^T x)(1-g(\theta^T x))$$

接下来，就剩下第三部分：

$$\frac{\partial \theta^T x}{\partial \theta_j} = \frac{\partial J(\theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n)}{\partial \theta_j} = x_j$$

综上所述：

$$\frac{\partial}{\partial \theta_j} J(\theta) = (y - h_{\theta}(x)) x_j$$

因此，梯度上升迭代公式为：

$$\theta_j := \theta_j + \alpha \sum_{i=1}^m (y^i - h_{\theta}(x^i)) x_j^i$$

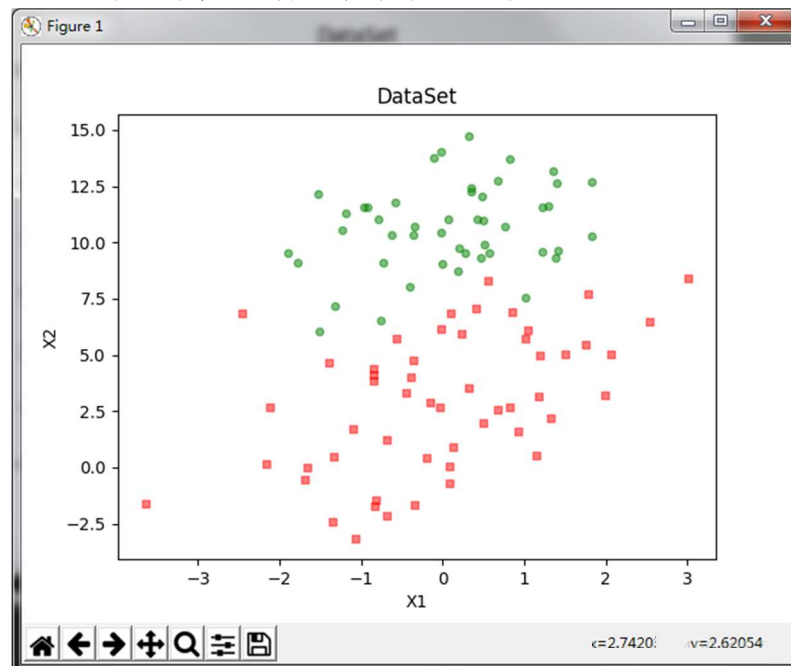
(3) 训练数据来自 testSet.txt。要求：

- 1)、使用梯度上升算法计算 θ ，输出 θ 并画出线性方程。
 - 2)、使用改进的随机梯度上升算法计算 θ ，输出 θ 并画出线性方程。
 - 3)、尝试将两种方式画出的线性方程画在同一个可视化视图上进行比较
- 先看下数据集有哪些数据：

1	-0.017612	14.053064	0
2	-1.395634	4.662541	1
3	-0.752157	6.538620	0
4	-1.322371	7.152853	0
5	0.423363	11.054677	0
6	0.406704	7.067335	1
7	0.667394	12.741452	0
8	-2.460150	6.866805	1
9	0.569411	9.548755	0
10	-0.026632	10.427743	0
11	0.850433	6.920334	1
12	1.347183	13.175500	0
13	1.176813	3.167020	1
14	-1.781871	9.097953	0
15	-0.566606	5.749003	1
16	0.931635	1.589505	1
17	-0.024205	6.151823	1
18	-0.036453	2.690988	1
19	-0.196949	0.444165	1
20	1.014459	5.754399	1
21	1.985298	3.230619	1
22	-1.693453	-0.557540	1
23	-0.576525	11.778922	0
24	-0.346811	-1.678730	1
25	-2.124484	2.672471	1
26	1.217916	9.597015	0
27	-0.733928	9.098687	0
28	-3.642001	-1.618087	1
29	0.315985	3.523953	1
30	1.416614	9.619232	0

这个数据有两维特征，因此可以将数据在一个二维平面上展示出来。我们可以将第一列数据(X_1)看作 x 轴上的值，第二列数据(X_2)看作 y 轴上的值。而最后一列数据即为分类标签。根据标签的不同，对这些点进行分类。

那么，我们可以编写代码，看下数据集的分布情况：



从上图可以看出数据的分布情况。假设 Sigmoid 函数的输入记为 z ，那么 $z = w_0x_0 + w_1x_1 + w_2x_2$ ，即可将数据分割开。其中， x_0 为全是 1 的向量， x_1 为数据

集的第一列数据， x_2 为数据集的第二列数据。另 $z=0$ ，则 $0=w_0 + w_1x_1 + w_2x_2$ 。横坐标为 x_1 ，纵坐标为 x_2 。这个方程未知的参数为 w_0, w_1, w_2 ，也就是我们要求的回归系数(最优参数)。

在编写代码之前，将梯度上升迭代公式矢量化：

$$\theta := \theta + \alpha X^T (\vec{y} - g(X\theta))$$

根据编写程序，运行结果如图所示：

```

148 if __name__ == '__main__':
149     dataMat, labelMat = loadDataSet()
150     print(gradAscent(dataMat, labelMat))

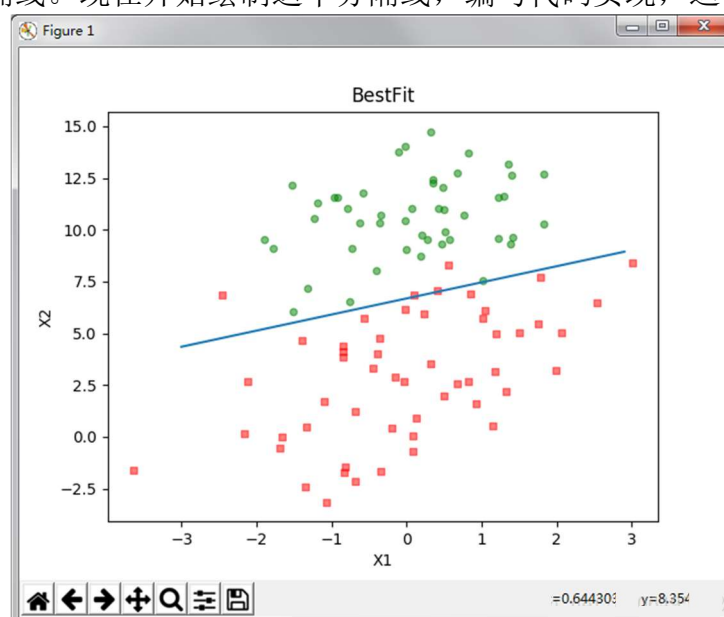
[[ 4.12414349]
 [ 0.48007329]
 [-0.6168482 ]]
[Finished in 1.2s]

```

可以看出，我们已经求解出回归系数 $[w_0, w_1, w_2]$ 。

通过求解出的参数，我们就可以确定不同类别数据之间的分隔线，画出决策边界。

下一步，绘制决策边界，我们已经解出了一组回归系数，它确定了不同类别数据之间的分隔线。现在开始绘制这个分隔线，编写代码实现，运行结果如下：



这个分类结果相当不错，从上图可以看出，只分错了几个点而已。但是，尽管例子简单数据集很小，但是这个方法却需要大量的计算(300 次乘法)。因此下篇文章将对改算法稍作改进，从而减少计算量，使其可以应用于大数据集上。

改进的随机梯度上升算法。梯度上升算法在每次更新回归系数(最优参数)时，都需要遍历整个数据集。可以看一下我们之前写的梯度上升算法：

```

def gradAscent(dataMatIn, classLabels):
    dataMatrix = np.mat(dataMatIn)

```

```

#转换成 numpy 的 mat
labelMat = np.mat(classLabels).transpose()
#转换成 numpy 的 mat,并进行转置
m, n = np.shape(dataMatrix)
#返回 dataMatrix 的大小。m 为行数,n 为列数。
alpha = 0.01
#移动步长,也就是学习速率,控制更新的幅度。
maxCycles = 500
#最大迭代次数
weights = np.ones((n,1))
for k in range(maxCycles):
    h = sigmoid(dataMatrix * weights)
#梯度上升矢量化公式
error = labelMat - h
weights = weights + alpha * dataMatrix.transpose() * error
return weights.getA(),weights_array

```

#将矩阵转换为数组，返回权重数组

假设，我们使用的数据集一共有 100 个样本。那么，dataMatrix 就是一个 100*3 的矩阵。每次计算 h 的时候，都要计算 dataMatrix*weights 这个矩阵乘法运算，要进行 100*3 次乘法运算和 100*2 次加法运算。同理，更新回归系数(最优参数)weights 时，也需要用到整个数据集，要进行矩阵乘法运算。总而言之，该方法处理 100 个左右的数据集时尚可，但如果有的数十亿样本和成千上万的特征，那么该方法的计算复杂度就太高了。因此，需要对算法进行改进，我们每次更新回归系数(最优参数)的时候，能不能不用所有样本呢？一次只用一个样本点去更新回归系数(最优参数)？这样就可以有效减少计算量了，这种方法就叫做随机梯度上升算法。

随机梯度上升算法

```

def stocGradAscent1(dataMatrix, classLabels, numIter=150):
    m,n = np.shape(dataMatrix)
#返回 dataMatrix 的大小。m 为行数,n 为列数。
    weights = np.ones(n)
#参数初始化
    for j in range(numIter):

```

```

        dataIndex = list(range(m))
        for i in range(m):
            alpha = 4/(1.0+j+i)+0.01
#降低 alpha 的大小，每次减小 1/(j+i)。
            randIndex = int(random.uniform(0,len(dataIndex)))
#随机选取样本
            h = sigmoid(sum(dataMatrix[randIndex]*weights))
#选择随机选取的一个样本，计算 h
            error = classLabels[randIndex] - h
#计算误差
            weights = weights + alpha * error * dataMatrix[randIndex]
#更新回归系数
            del(dataIndex[randIndex])
#删除已经使用的样本
        return weights
#返回

```

该算法第一个改进之处在于， α 在每次迭代的时候都会调整，并且，虽然 α 会随着迭代次数不断减小，但永远不会减小到 0，因为这里还存在一个常数项。必须这样做的原因是为了保证在多次迭代之后新数据仍然具有一定的影响。如果需要处理的问题是动态变化的，那么可以适当加大上述常数项，来确保新的值获得更大的回归系数。另一点值得注意的是，在降低 α 的函数中， α 每次减少 $1/(j+i)$ ，其中 j 是迭代次数， i 是样本点的下标。第二个改进的地方在于跟新回归系数(最优参数)时，只使用一个样本点，并且选择的样本点是随机的，每次迭代不使用已经用过的样本点。这样的方法，就有效地减少了计算量，并保证了回归效果。

编写代码如下，看下改进的随机梯度上升算法分类效果如何：

```

# -*- coding:UTF-8 -*-
from matplotlib.font_manager import FontProperties
import matplotlib.pyplot as plt
import numpy as np
import random

"""
函数说明:加载数据

```

```

Parameters:
    无
Returns:
    dataMat - 数据列表
    labelMat - 标签列表
"""
def loadDataSet():
    dataMat = [] # 创建数据列表
    labelMat = [] # 创建标签列表
    fr = open('testSet.txt') # 打开文件
    for line in fr.readlines():
        # 逐行读取
        lineArr = line.strip().split()
        # 去回车, 放入列表
        dataMat.append([1.0, float(lineArr[0]), float(lineArr[1])])
        # 添加数据
        labelMat.append(int(lineArr[2]))
        # 添加标签
    fr.close() # 关闭文件
    return dataMat, labelMat # 返回

"""
函数说明: sigmoid 函数

Parameters:
    inX - 数据
Returns:
    sigmoid 函数
"""
def sigmoid(inX):
    return 1.0 / (1 + np.exp(-inX))

"""
函数说明: 绘制数据集

Parameters:
    weights - 权重参数数组
Returns:
    无

```

```

"""
def plotBestFit(weights):
    dataMat, labelMat = loadDataSet()
    #加载数据集
    dataArr = np.array(dataMat)
    #转换成numpy的array数组
    n = np.shape(dataMat)[0] #
    #数据个数
    xcord1 = []; ycord1 = [] #
    #正样本
    xcord2 = []; ycord2 = [] #
    #负样本
    for i in range(n): #
        #根据数据集标签进行分类
        if int(labelMat[i]) == 1:
            xcord1.append(dataArr[i,1]);
            ycord1.append(dataArr[i,2]) #1 为正样本
        else:
            xcord2.append(dataArr[i,1]);
            ycord2.append(dataArr[i,2]) #0 为负样本
        fig = plt.figure()
        ax = fig.add_subplot(111)
        #添加 subplot
        ax.scatter(xcord1, ycord1, s = 20, c = 'red', marker =
's',alpha=.5) #绘制正样本
        ax.scatter(xcord2, ycord2, s = 20, c = 'green',alpha=.5)
        #绘制负样本
        x = np.arange(-3.0, 3.0, 0.1)
        y = (-weights[0] - weights[1] * x) / weights[2]
        ax.plot(x, y)
        plt.title('BestFit') #
    #绘制 title
    plt.xlabel('X1'); plt.ylabel('X2')
    #绘制 label
    plt.show()

```

"""

函数说明:改进的随机梯度上升算法

Parameters:

dataMatrix - 数据数组
classLabels - 数据标签
numIter - 迭代次数

Returns:

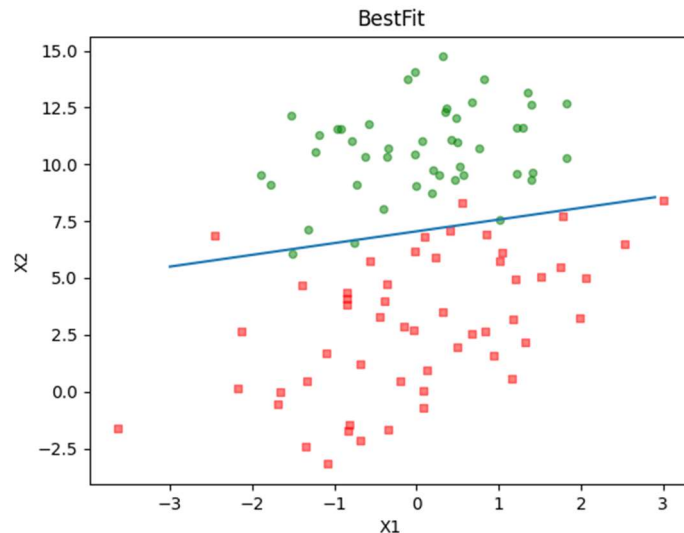

```

    weights - 求得的回归系数数组 (最优参数)
"""
def stocGradAscent1(dataMatrix, classLabels, numIter=150):
    m,n = np.shape(dataMatrix)
    #返回 dataMatrix 的大小。m 为行数,n 为列数。
    weights = np.ones(n)
    #参数初始化
    for j in range(numIter):
        dataIndex = list(range(m))
        for i in range(m):
            alpha = 4/(1.0+j+i)+0.01
            #降低 alpha 的大小, 每次减小 1/(j+i)。
            randIndex = int(random.uniform(0,len(dataIndex)))
            #随机选取样本
            h = sigmoid(sum(dataMatrix[randIndex]*weights))
            #选择随机选取的一个样本, 计算 h
            error = classLabels[randIndex] - h
            #计算误差
            weights = weights + alpha * error *
dataMatrix[randIndex]      #更新回归系数
            del (dataIndex[randIndex])
        #删除已经使用的样本
    return weights
#返回

if __name__ == '__main__':
    dataMat, labelMat = loadDataSet()
    weights = stocGradAscent1(np.array(dataMat), labelMat)
    plotBestFit(weights)

```

代码运行结果:



可以看到分类效果也是不错的。不过，从这个分类结果中，我们不好看出迭代次数和回归系数的关系，也就不能直观的看到每个回归方法的收敛情况。因此，我们编写程序，绘制出回归系数和迭代次数的关系曲线：

```
# -*- coding:UTF-8 -*-
from matplotlib.font_manager import FontProperties
import matplotlib.pyplot as plt
import numpy as np
import random

"""
函数说明:加载数据

Parameters:
    无
Returns:
    dataMat - 数据列表
    labelMat - 标签列表
"""
def loadDataSet():
    dataMat = [] #创建数据列表
    labelMat = [] #创建标签列表
    fr = open('testSet.txt') #打开文件
    for line in fr.readlines(): #逐行读取
        lineArr = line.strip().split() #去回车，放入列表
        dataMat.append([1.0, float(lineArr[0]), float(lineArr[1])])
        labelMat.append(int(lineArr[2])) #添加数据
    fr.close() #添加标签 #关闭文件
```

```

    return dataMat, labelMat                                #返回

"""
函数说明:sigmoid 函数

Parameters:
    inX - 数据
Returns:
    sigmoid 函数
"""
def sigmoid(inX):
    return 1.0 / (1 + np.exp(-inX))

"""
函数说明:梯度上升算法

Parameters:
    dataMatIn - 数据集
    classLabels - 数据标签
Returns:
    weights.getA() - 求得的权重数组(最优参数)
    weights_array - 每次更新的回归系数
"""
def gradAscent(dataMatIn, classLabels):
    dataMatrix = np.mat(dataMatIn)                        #转换成
    # numpy 的 mat
    labelMat = np.mat(classLabels).transpose()            #转换
    # 成 numpy 的 mat, 并进行转置
    m, n = np.shape(dataMatrix)                            #返回
    # dataMatrix 的大小。m 为行数, n 为列数。
    alpha = 0.01                                           #移动步长, 也就是学
    # 习速率, 控制更新的幅度。
    maxCycles = 500                                         #最大迭代次数
    weights = np.ones((n, 1))
    weights_array = np.array([])
    for k in range(maxCycles):
        h = sigmoid(dataMatrix * weights)                 #梯度上升
        # 矢量化公式
        error = labelMat - h
        weights = weights + alpha * dataMatrix.transpose() * error
        weights_array = np.append(weights_array, weights)
    weights_array = weights_array.reshape(maxCycles, n)
    return weights.getA(), weights_array                   #将矩
    # 阵转换为数组, 并返回

```

```
"""
```

函数说明:改进的随机梯度上升算法

Parameters:

dataMatrix - 数据数组
classLabels - 数据标签
numIter - 迭代次数

Returns:

weights - 求得的回归系数数组(最优参数)
weights_array - 每次更新的回归系数

```
"""
```

```
def stocGradAscent1(dataMatrix, classLabels, numIter=150):  
    m,n = np.shape(dataMatrix) #返回  
    dataMatrix 的大小。m 为行数,n 为列数。  
    weights = np.ones(n) #参数初  
    始化  
    weights_array = np.array([]) #存储每次  
    更新的回归系数  
    for j in range(numIter):  
        dataIndex = list(range(m))  
        for i in range(m):  
            alpha = 4/(1.0+j+i)+0.01 #降低  
            alpha 的大小, 每次减小 1/(j+i)。  
            randIndex = int(random.uniform(0,len(dataIndex)))  
            #随机选取样本  
            h = sigmoid(sum(dataMatrix[dataIndex[randIndex]]*weights))  
            #选择随机选取的一个样本, 计算 h  
            error = classLabels[dataIndex[randIndex]] - h  
            #计算误差  
            weights = weights + alpha * error *  
            dataMatrix[dataIndex[randIndex]] #更新回归系数  
            weights_array = np.append(weights_array,weights,axis=0)  
            #添加回归系数到数组中  
            del (dataIndex[randIndex]) #删除已经  
            使用的样本  
    weights_array = weights_array.reshape(numIter*m,n)  
    #改变维度  
    return weights,weights_array #返回
```

```
"""
```

函数说明:绘制数据集

Parameters:

```

    weights - 权重参数数组
Returns:
    无
"""
def plotBestFit(weights):
    dataMat, labelMat = loadDataSet()           #加载数据集
    dataArr = np.array(dataMat)                 #转换成numpy的array数组
    n = np.shape(dataMat)[0]                    #数据个数
    xcord1 = []; ycord1 = []                   #正样本
    xcord2 = []; ycord2 = []                   #负样本
    for i in range(n):                         #根据数据集标签进行分类
        if int(labelMat[i]) == 1:
            xcord1.append(dataArr[i,1]); ycord1.append(dataArr[i,2])
        #1 为正样本
        else:
            xcord2.append(dataArr[i,1]); ycord2.append(dataArr[i,2])
        #0 为负样本
    fig = plt.figure()
    ax = fig.add_subplot(111)                  #添加subplot
    ax.scatter(xcord1, ycord1, s = 20, c = 'red', marker = 's', alpha=.5) #绘制正样本
    ax.scatter(xcord2, ycord2, s = 20, c = 'green', alpha=.5) #绘制负样本
    x = np.arange(-3.0, 3.0, 0.1)
    y = (-weights[0] - weights[1] * x) / weights[2]
    ax.plot(x, y)
    plt.title('BestFit')                      #绘制title
    plt.xlabel('X1'); plt.ylabel('X2')        #绘制label
    plt.show()

"""
函数说明:绘制回归系数与迭代次数的关系

Parameters:
    weights_array1 - 回归系数数组 1
    weights_array2 - 回归系数数组 2
Returns:
    无
"""

```

```

def plotWeights(weights_array1, weights_array2):
    # 设置汉字格式
    font = FontProperties(fname=r"c:\windows\fonts\simSun.ttc",
size=14)
    # 将 fig 画布分隔成 1 行 1 列, 不共享 x 轴和 y 轴, fig 画布的大小为 (13, 8)
    # 当 nrow=3, nclos=2 时, 代表 fig 画布被分为六个区域, axs[0][0] 表示第一行第一列
    fig, axs = plt.subplots(nrows=3, ncols=2, sharex=False,
sharey=False, figsize=(20, 10))
    x1 = np.arange(0, len(weights_array1), 1)
    # 绘制 w0 与迭代次数的关系
    axs[0][0].plot(x1, weights_array1[:, 0])
    axs0_title_text = axs[0][0].set_title(u'梯度上升算法: 回归系数与迭代次数关系', fontproperties=font)
    axs0_ylabel_text =
axs[0][0].set_ylabel(u'W0', fontproperties=font)
    plt.setp(axs0_title_text, size=20, weight='bold',
color='black')
    plt.setp(axs0_ylabel_text, size=20, weight='bold',
color='black')
    # 绘制 w1 与迭代次数的关系
    axs[1][0].plot(x1, weights_array1[:, 1])
    axs1_ylabel_text =
axs[1][0].set_ylabel(u'W1', fontproperties=font)
    plt.setp(axs1_ylabel_text, size=20, weight='bold',
color='black')
    # 绘制 w2 与迭代次数的关系
    axs[2][0].plot(x1, weights_array1[:, 2])
    axs2_xlabel_text = axs[2][0].set_xlabel(u'迭代次数', fontproperties=font)
    axs2_ylabel_text =
axs[2][0].set_ylabel(u'W1', fontproperties=font)
    plt.setp(axs2_xlabel_text, size=20, weight='bold',
color='black')
    plt.setp(axs2_ylabel_text, size=20, weight='bold',
color='black')

    x2 = np.arange(0, len(weights_array2), 1)
    # 绘制 w0 与迭代次数的关系
    axs[0][1].plot(x2, weights_array2[:, 0])
    axs0_title_text = axs[0][1].set_title(u'改进的随机梯度上升算法: 回归系数与迭代次数关系', fontproperties=font)
    axs0_ylabel_text =

```

```

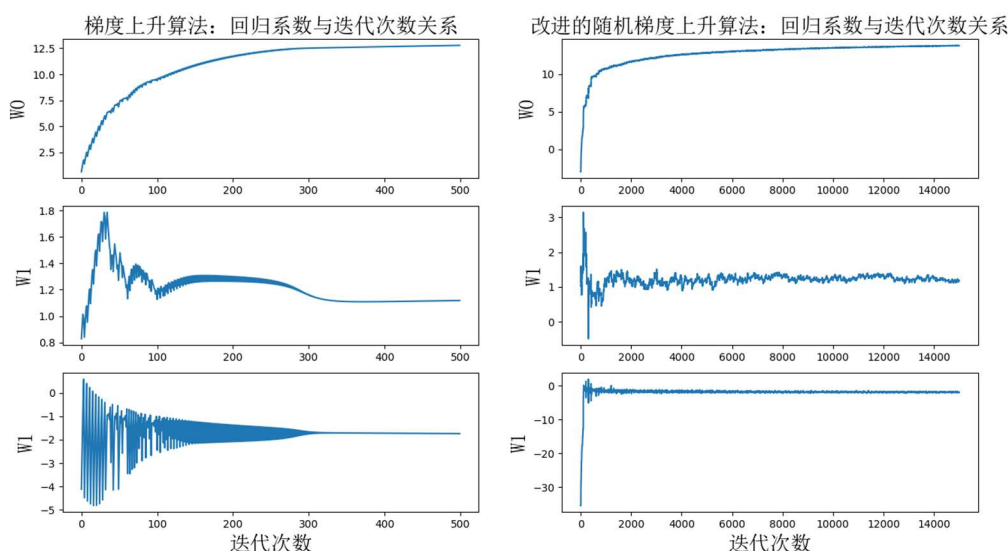
axs[0][1].set_ylabel(u'W0',fontproperties=font)
plt.setp(axs0_title_text, size=20, weight='bold',
color='black')
plt.setp(axs0_ylabel_text, size=20, weight='bold',
color='black')
#绘制 w1 与迭代次数的关系
axs[1][1].plot(x2,weights_array2[:,1])
axs1_ylabel_text =
axs[1][1].set_ylabel(u'W1',fontproperties=font)
plt.setp(axs1_ylabel_text, size=20, weight='bold',
color='black')
#绘制 w2 与迭代次数的关系
axs[2][1].plot(x2,weights_array2[:,2])
axs2_xlabel_text = axs[2][1].set_xlabel(u'迭代次数
',fontproperties=font)
axs2_ylabel_text =
axs[2][1].set_ylabel(u'W1',fontproperties=font)
plt.setp(axs2_xlabel_text, size=20, weight='bold',
color='black')
plt.setp(axs2_ylabel_text, size=20, weight='bold',
color='black')

plt.show()

if __name__ == '__main__':
    dataMat, labelMat = loadDataSet()
    weights1, weights_array1 = stocGradAscent1(np.array(dataMat),
labelMat)

    weights2, weights_array2 = gradAscent(dataMat, labelMat)
    plotWeights(weights_array2, weights_array1)

```



由于改进的随机梯度上升算法，随机选取样本点，所以每次的运行结果是不同的。但是大体趋势是一样的。我们改进的随机梯度上升算法收敛效果更好。为什么这么说呢？让我们分析一下。我们一共有 100 个样本点，改进的随机梯度上升算法迭代次数为 150。而上图显示 15000 次迭代次数的原因是，使用一次样本就更新一下回归系数。因此，迭代 150 次，相当于更新回归系数 $150 \times 100 = 15000$ 次。简而言之，迭代 150 次，更新 1.5 万次回归参数。从上图左侧的改进随机梯度上升算法回归效果中可以看出，其实在更新 2000 次回归系数的时候，已经收敛了。相当于遍历整个数据集 20 次的时候，回归系数已收敛。训练已完成。

再让我们看看上图右侧的梯度上升算法回归效果，梯度上升算法每次更新回归系数都要遍历整个数据集。从图中可以看出，当迭代次数为 300 多次的时候，回归系数才收敛。凑个整，就当它在遍历整个数据集 300 次的时候已经收敛好了。

改进的随机梯度上升算法，在遍历数据集的第 20 次开始收敛。而梯度上升算法，在遍历数据集的第 300 次才开始收敛。

四：实验要求

- 1、根据实验讲义完成相应实验内容，对比实验结果，调整参数，保证结果的正确性；
- 2、记录实验过程，撰写电子版实验报告，注意每张结果图片上需注明同学们的“学号+姓名”；
- 3、完成电子版实验报告后，调整报告格式内容，不限于字体、字号、标题、段落、图片大小等等，保证报告结果正确且美观，检查无误在学习通平台提交电子版实验报告文件，提交前修改电子版文件名称为“学号+姓名+实验 3”；
- 4、打印纸质版实验报告进行保存，所有实验结束后整体装订成册，交给各班学习委员。